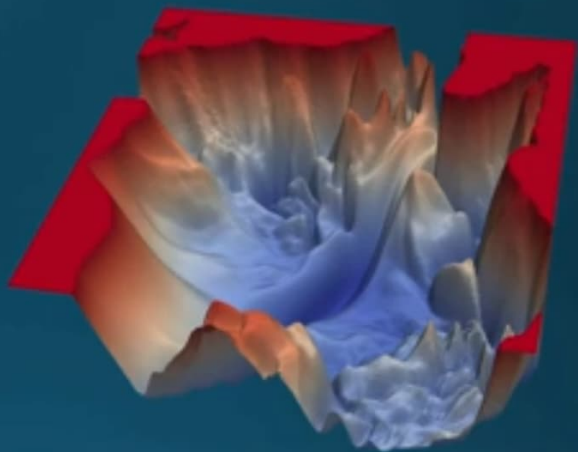
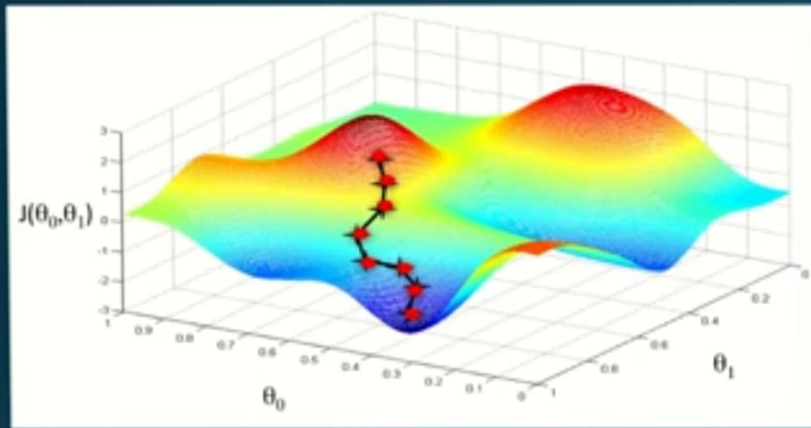


Gradient Descent challenges

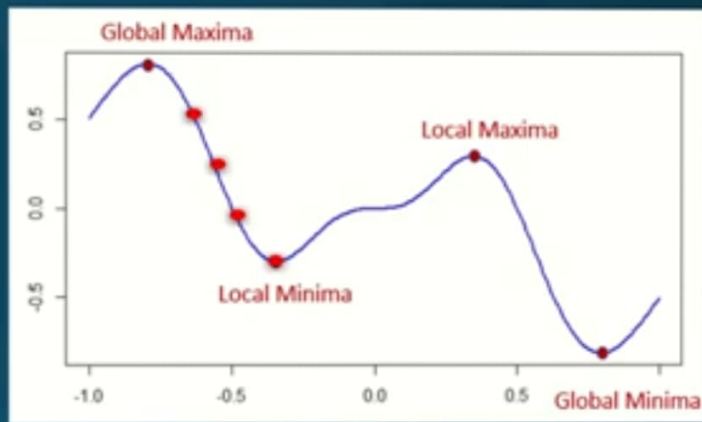
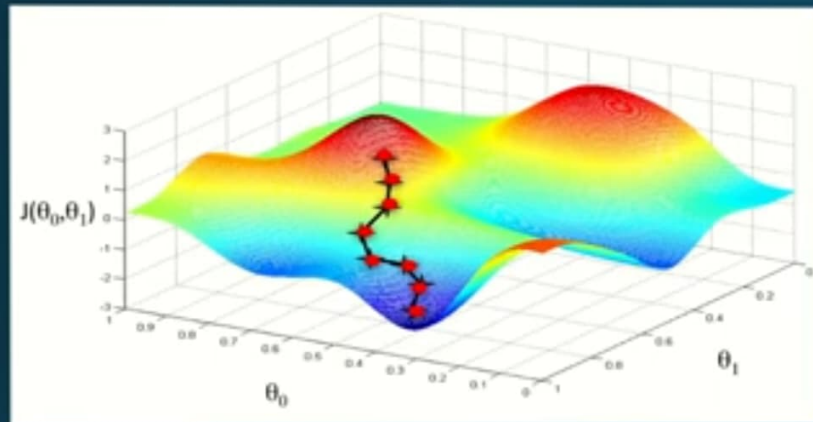


Error surface of neural networks is simply a plot of all your parameters of the neural network versus the corresponding cost value or the loss function value

Modern neural networks that are used in practice especially in computer vision have millions of parameters

Gradient Descent Goal?

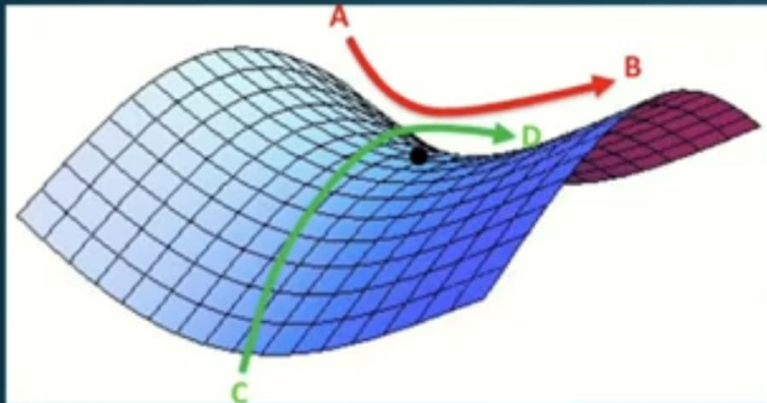
Gradient Descent challenges



Non-convex surface

When you hit one of this local minima, your gradient is going to be zero and your gradient descent algorithm will terminate.

Gradient Descent challenges

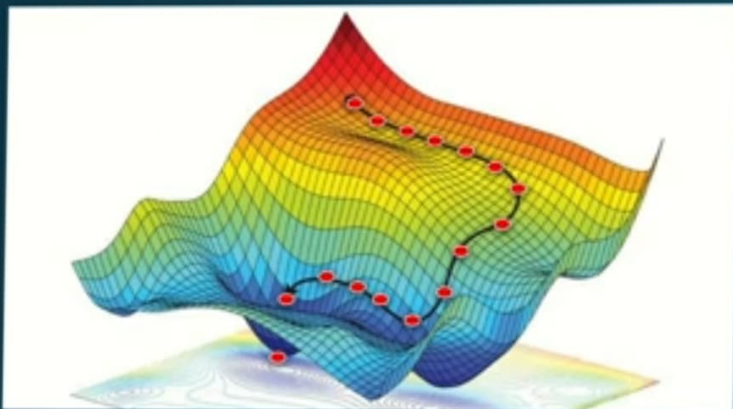


Saddle points - minima along a set of dimensions and maxima along another set of dimensions

Gradient is zero at a saddle point

Saddle points proliferate as you go to higher and higher dimensions

Gradient Descent challenges

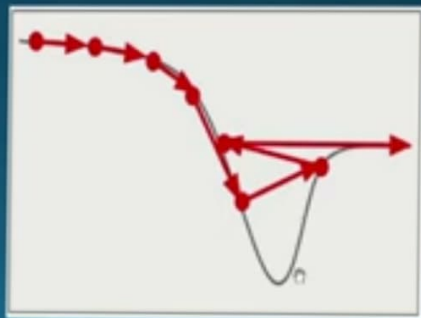


Yes, we need something different!!!

OBSERVATION: There are different points at which the speed of gradient descent can keep changing.

Flat regions: Gradient values are small; the parameter updates are going to be small

Can we speed up by increasing learning rate?



Flat regions: Traversal will be faster

Valleys: Risk of convergence

Momentum based gradient descent

Why should we rely only on the gradient at the current time step?

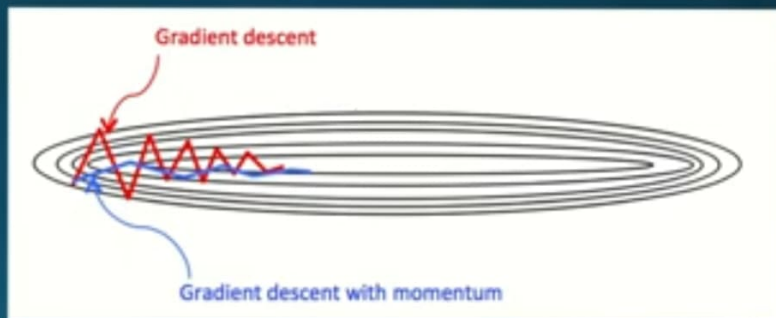
Why not we make use of the direction that we have been traversing on so far to make our next update?

With increasing confidence increase the step size and with decreasing confidence decrease the step size

$$v_t = \underbrace{\gamma v_{t-1}}_{\text{Momentum Term}} + \alpha \nabla_{\theta_t} \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})$$
$$\theta_{t+1} = \theta_t - v_t$$

γ lies between 0 and 1

Larger the γ more the previous gradients affect the current step



General practice is to start γ with 0.5 until your initial learning stabilizes

Gradient stabilizes after some time then you can increase your momentum parameter γ to 0.9 or 0.95

Challenges

Reaches the region of the minima, then keeps oscillating and finally gets to the minimum

It converges much faster than vanilla

GD

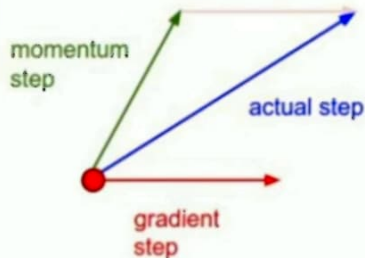
Nonetheless, we see that momentum seems to be wasting some time in oscillating around that area

Can we improve??

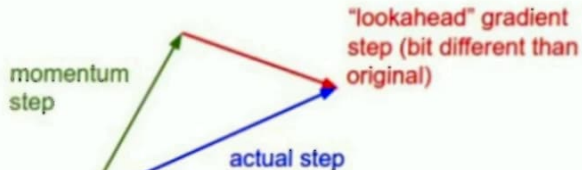
Nesterov Accelerated Momentum

It has produced slightly better results than classical Momentum.

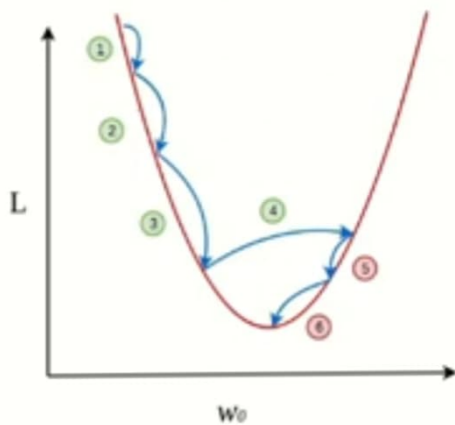
Momentum update



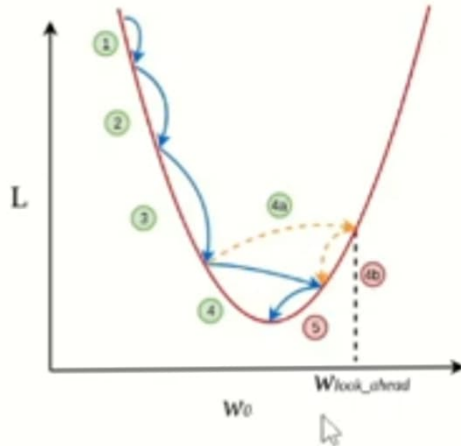
Nesterov momentum update



NAG resolves this problem by adding a look ahead term in our equation. The intuition behind NAG can be summarized as 'look before you leap'. Let us try to understand this through an example.



(a) Momentum-Based Gradient Descent



(b) Nesterov Accelerated Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

Gradient Descent Types

Batch Gradient Descent

For every parameter update gradient descent passes the entire dataset..

If your training dataset have 1 million data points, you are going to wait for passing through all of them before making an update

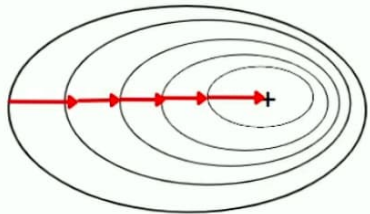
Stochastic Gradient Descent

If you randomly shuffle and pick one data point, forward propagate it, compute the error, compute the gradient, update all your parameters right away before propagating your next data point.

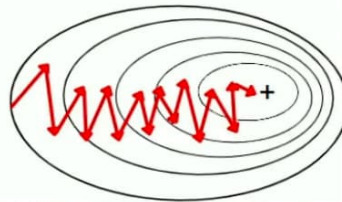
Mini batch Gradient Descent

you take a mini batch of training examples, do not just take one, do not take all, take 20 examples from your training dataset, forward propagate all of them, compute all the outputs, compute all the errors, compute all the gradients and then taking average of those 20 gradients alone and update your parameters

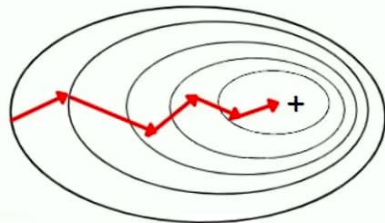
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



Gradient Descent Types

Stochastic gradient descent is a very popular and common algorithm used in various Machine Learning algorithms

Stochastic gradient descent help you in escaping local minima, bad local minima or saddle points

Can we control "Stochasticity" of SGD?? → **Learning rate**

How do we choose Learning rate?

Learning rate constant throughout the process- **Naïve linear search**

Methods where you gradually reduce the learning rate over time- **Annealing based methods**

You reduce the learning rate after every n iterations or if certain degenerative conditions are met(if the current error is more than the previous error)- **Step decay**

You set an initial learning rate called α where $\alpha = \alpha^{-kt}$, α and k are hyper parameters and t is an iteration number- **Exponential decay**

Can we do something better??

- Be more aware of what a neural network is doing and accordingly choose a learning rate rather than have these global rules that we fix irrespective of what the network is doing

Adaptive gradient (adagrad)

- Gradient is a vector of the derivatives of the loss with respect to every weight in your neural network
- Why should you use the same learning rate for all of them?
- Why can't you use a different learning rate for each of these weights?

$$r_t = r_{t-1} + (\Delta\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\delta + \sqrt{r_t}} \Delta\theta_t$$

Adagrad - you accumulate the squared gradients as a running sum(r_t)

Delta - Used for numerical stability

Gradient was large -give it a small learning rate

Gradient is small- give it a larger learning rate

RMSProp

Problem with Adagrad

Gradient accumulation term r is a running sum that simply keeps t accumulating over iterations, which means as you run more and more iterations, it will only keep increasing and get higher and higher and higher over time which means this denominator is definitely going to get high over time for all the weights

$$r_t = (\rho)r_{t-1} + (1 - \rho)(\Delta\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\delta + \sqrt{r_t}} \Delta\theta_t$$

ρ and $1 - \rho$, the sum cannot keep exploding and is now controlled

Adaptive Moments(ADAM)

The most popular algorithm that is used for training neural networks today for adapting learning rates

Adaptive moments uses a simple intuition of combining ideas of RMSProp and momentum

$$s_t = (\rho_1)r_{t-1} + (1 - \rho_1)(\Delta\theta_t)$$

$$r_t = (\rho_2)r_{t-1} + (1 - \rho_2)(\Delta\theta_t)^2$$

Bias Correction: $\tilde{s}_t = \frac{s_t}{1 - \rho_1^t}, \tilde{r}_t = \frac{r_t}{1 - \rho_2^t}$

$$\theta_{t+1} = \theta_t - \alpha \frac{\tilde{s}_t}{\delta + \sqrt{\tilde{r}_t}}$$

ρ_1 is the momentum coefficient(value~~ 0.9)